

API AURORA

Documentation technique et guide débutant

Version : 2026-09-25

Python / Flask — Administration Linux

1. Présentation

L'API AURORA est une API HTTP développée en Python avec Flask. Elle permet d'administrer à distance un serveur Linux.

Elle permet notamment de configurer le réseau, le DNS et le NTP, modifier le hostname et /etc/hosts, redémarrer ou arrêter le serveur, lancer des opérations APT et redémarrer des services.

⚠ Cette API possède des droits d'administration importants. Elle doit être protégée et ne pas être exposée inutilement à Internet.

2. Informations générales

Élément	Valeur
Technologie	Python / Flask
Adresse d'écoute	0.0.0.0
Port	55001
Protocole actuel	HTTP
Authentification	Bearer Token

Exemple d'URL de base :

```
http://192.168.1.100:55001
```

3. Authentification

Toutes les routes protégées nécessitent cet en-tête HTTP :

```
Authorization: Bearer supersecrettoken123
```

Le token est actuellement écrit directement dans le programme Python. Il est recommandé de le remplacer par une valeur longue et aléatoire.

4. Format des requêtes

Tous les endpoints utilisent actuellement POST. Les paramètres sont envoyés en JSON lorsque nécessaire.

```
POST http://192.168.1.100:55001/set-hostname
```

```
Content-Type: application/json
```

5. Codes HTTP

Code	Signification
200	Opération acceptée / réussie
400	Paramètre obligatoire manquant
403	Token absent ou incorrect
500	Erreur interne / commande système échouée

6. /set-network

6.1 DHCP

Configure une interface en DHCP puis exécute netplan apply.

```
curl -X POST http://192.168.1.100:55001/set-network \
-H "Authorization: Bearer supersecrettoken123" \
-H "Content-Type: application/json" \
-d '{"interface":"eth0","dhcp":true}'

{"status":"ok","mode":"dhcp"}
```

6.2 IP statique

```
curl -X POST http://192.168.1.100:55001/set-network \
-H "Authorization: Bearer supersecrettoken123" \
-H "Content-Type: application/json" \
-d '{
  "interface":"eth0",
  "dhcp":false,
  "ip":"192.168.1.100",
  "prefix":24,
  "gateway":"192.168.1.1",
  "dns":["1.1.1.1","8.8.8.8"]
}'

{"status":"ok","mode":"static"}
```

Paramètre	Obligatoire	Exemple	Description
interface	Non	eth0	Interface réseau
dhcp	Non	false	Active DHCP
ip	Si dhcp=false	192.168.1.100	Adresse IP
prefix	Si dhcp=false	24	Préfixe CIDR
gateway	Si dhcp=false	192.168.1.1	Passerelle
dns	Non	["1.1.1.1"]	Serveurs DNS

⚠ Un changement d'IP peut couper immédiatement la connexion à l'API.

7. /set-hostname

Modifie le nom du serveur avec hostnamectl.

```
curl -X POST http://192.168.1.100:55001/set-hostname \
-H "Authorization: Bearer supersecrettoken123" \
-H "Content-Type: application/json" \
-d '{"hostname":"aurora-01"}'

{"status":"ok","hostname":"aurora-01"}
```

8. /set-dns

Configure uniquement les DNS d'une interface et applique Netplan.

```
curl -X POST http://192.168.1.100:55001/set-dns \
-H "Authorization: Bearer supersecrettoken123" \
```

```
-H "Content-Type: application/json" \  
-d '{"interface":"eth0","dns":["1.1.1.1","8.8.8.8"]}'  
  
{"status":"ok","dns":["1.1.1.1","8.8.8.8"]}
```

9. /set-ntp

Configure `/etc/systemd/timesyncd.conf` puis active et redémarre `systemd-timesyncd`.

```
curl -X POST http://192.168.1.100:55001/set-ntp \  
-H "Authorization: Bearer supersecrettoken123" \  
-H "Content-Type: application/json" \  
-d '{"servers":["0.fr.pool.ntp.org","1.fr.pool.ntp.org"]}'  
  
{"status":"ok","ntp":["0.fr.pool.ntp.org","1.fr.pool.ntp.org"]}  
  
[Time]  
NTP=0.fr.pool.ntp.org 1.fr.pool.ntp.org
```

10. /reboot

Redémarre normalement le serveur.

```
curl -X POST http://192.168.1.100:55001/reboot \  
-H "Authorization: Bearer supersecrettoken123"  
  
{"status":"rebooting"}
```

11. /forcereboot

Effectue un redémarrage forcé avec `systemctl reboot --force --force`.

```
curl -X POST http://192.168.1.100:55001/forcereboot \  
-H "Authorization: Bearer supersecrettoken123"  
  
{"status":"rebooting"}
```

⚠ Opération beaucoup plus brutale qu'un redémarrage normal.

12. /shutdown

Arrête complètement le serveur.

```
curl -X POST http://192.168.1.100:55001/shutdown \  
-H "Authorization: Bearer supersecrettoken123"  
  
{"status":"shutting down"}
```

13. /update

Lance en arrière-plan : `apt-get upgrade --fix-missing -y`.

```
curl -X POST http://192.168.1.100:55001/update \  
-H "Authorization: Bearer supersecrettoken123"
```

```
{"status":"update started"}
```

Important : cet endpoint ne lance pas `apt-get update` ; il lance une commande `upgrade`.

14. /upgrade

Lance en arrière-plan : `apt-get upgrade -y`.

```
curl -X POST http://192.168.1.100:55001/upgrade \  
-H "Authorization: Bearer supersecrettoken123" \  
  
{"status":"upgrade started"}
```

15. /set-hosts

Réécrit `/etc/hosts` avec une base standard puis ajoute les entrées fournies.

```
curl -X POST http://192.168.1.100:55001/set-hosts \  
-H "Authorization: Bearer supersecrettoken123" \  
-H "Content-Type: application/json" \  
-d '{  
  "entries":[  
    {"ip":"192.168.1.10","hostname":"serveur-db"},  
    {"ip":"192.168.1.20","hostname":"serveur-web"}  
  ]  
'
```

⚠ Les anciennes entrées personnalisées non renvoyées dans la requête sont supprimées.

16. /restart-service

Redémarre un service Linux avec `service NOM_DU_SERVICE restart`.

```
curl -X POST http://192.168.1.100:55001/restart-service \  
-H "Authorization: Bearer supersecrettoken123" \  
-H "Content-Type: application/json" \  
-d '{"service":"apache2"}'  
  
{"status":"ok","service":"apache2"}
```

Exemple MariaDB :

```
{"service":"mariadb"}
```

Sans service :

```
{"error":"Missing service parameter"}
```

17. Tableau récapitulatif

Endpoint	Méthode	JSON	Fonction
/set-network	POST	Oui	Configure le réseau
/set-hostname	POST	Oui	Modifie le hostname
/set-dns	POST	Oui	Configure les DNS
/set-ntp	POST	Oui	Configure le NTP

/reboot	POST	Non	Redémarre
/forcereboot	POST	Non	Redémarrage forcé
/shutdown	POST	Non	Arrête
/update	POST	Non	apt-get upgrade --fix-missing
/upgrade	POST	Non	apt-get upgrade
/set-hosts	POST	Oui	Modifie /etc/hosts
/restart-service	POST	Oui	Redémarre un service

18. Utilisation avec Python

```
import requests

url = "http://192.168.1.100:55001/set-hostname"
headers = {
    "Authorization": "Bearer supersecrettoken123",
    "Content-Type": "application/json"
}
data = {"hostname": "aurora-01"}

response = requests.post(url, headers=headers, json=data)
print(response.status_code)
print(response.json())
```

19. Utilisation avec Postman

- Créer une requête.
- Sélectionner POST.
- Entrer l'URL.
- Ajouter Authorization: Bearer TOKEN dans Headers.
- Ajouter Content-Type: application/json.
- Pour les paramètres, choisir Body > raw > JSON.
- Saisir le JSON puis cliquer sur Send.

20. Journalisation

Les logs sont écrits dans /opt/docker/Aurora/scripts/log/. Le fichier change chaque jour.

```
/opt/docker/Aurora/scripts/log/log-API_AURORA_2026-09-25.log

2026-09-25 10:15:32 [INFO] Hostname changed to aurora-01

2026-09-25 10:16:10 [INFO] Set eth0 to DHCP
```

21. Vérifier le fonctionnement

```
ss -lntp | grep 55001

curl -X POST http://127.0.0.1:55001/set-hostname \
-H "Authorization: Bearer supersecrettoken123" \
-H "Content-Type: application/json" \
-d '{"hostname": "test-aurora"}'
```

22. Erreurs courantes

22.1 HTTP 403

```
{"error": "Unauthorized"}
```

Token absent, incorrect ou mal formaté.

22.2 HTTP 400

Paramètre obligatoire manquant, par exemple service pour /restart-service.

22.3 HTTP 500

Une commande système ou une opération Python a échoué. Consulter les logs.

23. Sécurité

- Utiliser HTTPS pour protéger le token.
- Remplacer le token par une valeur longue et aléatoire.
- Limiter le port 55001 au réseau d'administration.
- Éviter une exposition directe sur Internet.
- Surveiller les logs.

24. Architecture simplifiée

```
Client
|
| HTTP POST + Bearer Token
v
API AURORA
Flask / Python
Port 55001
|
+--> Netplan
+--> hostnamectl
+--> systemd-timesyncd
+--> reboot / shutdown
+--> apt-get
+--> /etc/hosts
+--> services Linux
```

25. Points importants pour les débutants

- Tous les endpoints utilisent POST.
- Le token est obligatoire.
- Les paramètres sont généralement envoyés en JSON.
- Une modification réseau peut couper la connexion.
- reboot et shutdown interrompent le serveur.
- forcereboot est une opération très brutale.
- Les commandes APT sont lancées en arrière-plan.
- HTTP 200 ne signifie pas forcément qu'une commande en arrière-plan est terminée.
- En cas de HTTP 500, consulter les logs.